Program-5

Application of Convolutional Neural Network - 1

Aim: Build a Convolutional Neural Network for MNIST Handwritten Digit Classification.

Introduction to MNIST Dataset:

The MNIST (Modified National Institute of Standards and Technology) dataset is a very popular dataset used in the field of computer vision and deep learning. Because of its simplicity and widespread use for introducing and testing new algorithms.

The MNIST dataset is a large collection of **handwritten digits**. The digits are standardized to a fixed size and are represented as grayscale images.

- **Size**: The dataset contains 70,000 images in total.
  - **Training Set**: 60,000 images for training a model.
  - **Test Set**: 10,000 images for evaluating the trained model.
- **Image Dimensions**: Each image is a **28x28 pixel grayscale image**. Since it's grayscale, each pixel has a single value representing its intensity, typically ranging from 0 (black) to 255 (white).
- **Labels**: Each image has a corresponding label, which is an integer from 0 to 9, indicating the digit it represents.

Introduction to Convolutional Neural Networks:

A Convolutional Neural Network (CNN) is a type of deep learning model that is exceptionally good at processing image data. When applied to the MNIST dataset, a CNN is used to automatically learn the features of handwritten digits and classify them with high accuracy.

A CNN processes the image in its original 2D, or 3D form. It uses a series of specialized layers to extract features.

- **Convolutional Layers**:
  - These are the core of CNN. They scan the input image with small filters to detect features like edges, curves, and corners.
  - In the case of MNIST, the first layer might learn to identify simple lines, and subsequent layers learn to combine these lines to form shapes that look like parts of a digit.
- **Pooling Layers**:
  - After a convolutional layer, a pooling layer is often used to down sample the feature maps.
  - This reduces the spatial dimensions of the data, which makes the model more computationally efficient and more robust to small shifts or distortions in the handwritten digits.
- **Fully Connected Layers**:
  - After several convolutional and pooling layers have extracted high-level features, the output is flattened into a one-dimensional vector.

- This vector is then fed into a standard neural network layer (or layers) that performs the final classification.

For the MNIST dataset, the final layer of the CNN has 10 neurons, one for each digit from 0 to 9. A softmax activation function is applied to this layer to output a probability distribution, where the sum of the probabilities is 1. The neuron with the highest probability corresponds to the model's prediction of the digit in the input image.

Modules used:

1. Tensorflow
2. Keras
3. Matplotlib : for graphical visualization of result
4. to_categorical : for one-hot encoding of the image labels

Steps involved:

1. **Data preparation** involves loading the data, splitting the data into training and testing, reshaping the image (no. of samples, height, width and channel), normalizing the pixel values and one-hot encoding of the image labels.
2. **Model building** involves creating a stack of layers viz. convolution and pooling layers, flattening of layers and interconnecting with all the neurons in the final layer (dense).
3. **Training and evaluation** involve choosing the best optimizer, loss function, metrics like accuracy for monitoring. In addition, the model performance is plotted during the training over time.

Program:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
#Step-1 Data Preparation - Pixel values
#Load the data and split into training and testing data
(x_train, y_train), (x_test, y_test) = mnist.load_data()
#Reshape the image to 4-D format, including the channel dimension
x_dt = x_test.reshape ((x_test.shape[0]),28,28,1)
#Normalize the pixel values to the range [0,1]
x_train = x_train.astype('float32')/255
x_test = x_test.astype('float32')/255
#Perform one-hot encoding to convert labels to categorical (0/1)
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
#Step-2 Build the CNN model
```

```
model = models.Sequential()
# First convolutional layer
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))
# Second convolutional layer
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
# Third convolutional layer
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
# Flatten the output to feed it into a dense layer
model.add(layers.Flatten())
# Fully connected layer
model.add(layers.Dense(64, activation='relu'))
# Output layer with softmax activation for classification
model.add(layers.Dense(10, activation='softmax'))
# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
#Step-3 Training and Evaluation
#Train the model
history = model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.2)
# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")
# Plot training & validation accuracy values
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.savefig('Model Accuracy.tiff')
plt.show()
# Plot training & validation loss values
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
```

```
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.savefig('Model loss.tiff')
plt.show()
```

Output: Test accuracy: 0.9888

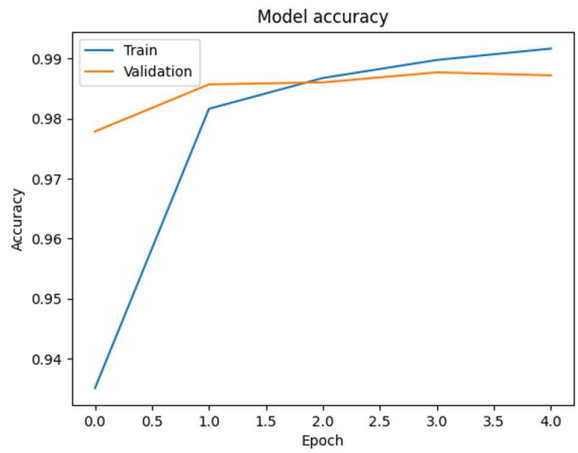

Fig. 1 Model Accuracy plot

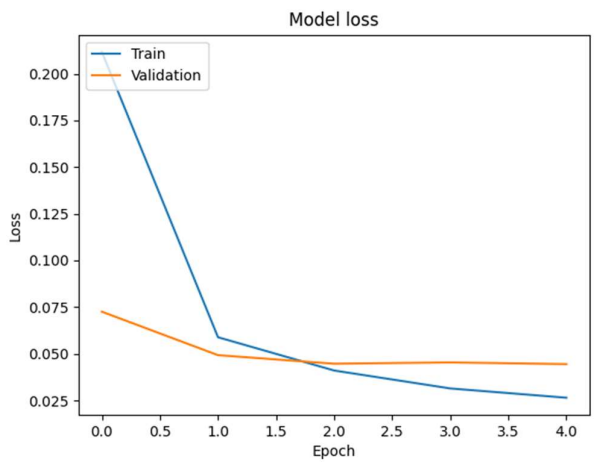

Fig. 2 Model Loss plot

Program-6

Application of Convolutional Neural Network - 2

Aim: Build a Convolutional Neural Network for Simple Image Classification.

Introduction to MNIST Dataset:

The CIFAR-10 dataset is a collection of small, low-resolution color images. It contains 60,000 images, each sized 32x32 pixels with 3 color channels (RGB). The images are categorized into 10 distinct classes. There are 6,000 images per class. The 10 classes are:

S.No.	Class	S.No.	Class
1	Airplane	6	Dog
2	Automobile	7	Frog
3	Bird	8	Horse
4	Cat	9	Ship
5	Deer	10	Truck

Modules used:

1. Tensorflow
2. Keras
3. Matplotlib : for graphical visualization of result
4. to_categorical : for one-hot encoding of the image labels

Program:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10
import matplotlib.pyplot as plt
# Load and preprocess the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
# Normalize the pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
# Convert labels to categorical (one-hot encoding)
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
# Build the CNN model
model = models.Sequential()
# First convolutional layer
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
```

```
# Second convolutional layer
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
# Third convolutional layer
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
# Flatten the output to feed into a dense layer
model.add(layers.Flatten())
# Fully connected layer
model.add(layers.Dense(128, activation='relu'))
# Output layer with softmax activation for classification (10 classes for CIFAR-10)
model.add(layers.Dense(10, activation='softmax'))
# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
# Train the model
history = model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.2)
# Evaluate the model on the test data
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")
# Plot training & validation accuracy values
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
# Plot training & validation loss values
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
```

```
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

Output: Test accuracy: 0.6978

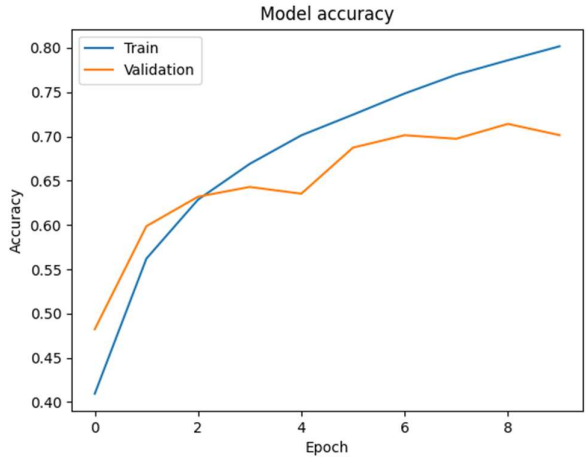

Fig. 1 Model Accuracy plot

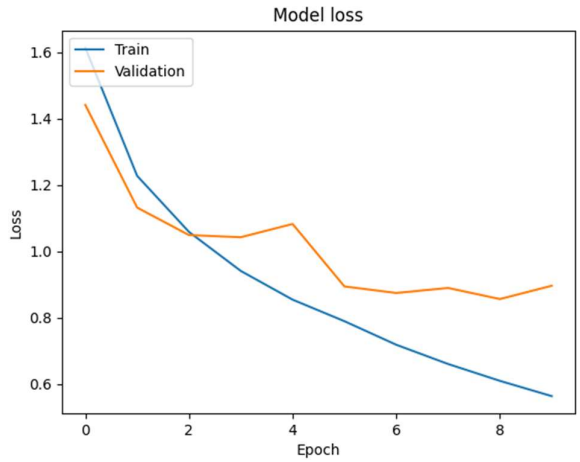

Fig. 2 Model Loss plot